CENTRO UNIVERSITÁRIO PARA O DESENVOLVIMENTO DO ALTO VALE DO ITAJAÍ - UNIDAVI

GUSTAVO BORGONHA

SISTEMA DE APOIO À ESTIMATIVA DE ESFORÇO EM DEMANDAS DE BUSINESS INTELLIGENCE BASEADO EM DADOS HISTÓRICOS E REGRESSÃO

RIO DO SUL
2026

CENTRO UNIVERSITÁRIO PARA O DESENVOLVIMENTO DO ALTO VALE DO ITAJAÍ - UNIDAVI

GUSTAVO BORGONHA

SISTEMA DE APOIO À ESTIMATIVA DE ESFORÇO EM DEMANDAS DE BUSINESS INTELLIGENCE BASEADO EM DADOS HISTÓRICOS E REGRESSÃO

Trabalho de Conclusão de Curso a ser apresentado ao curso de Sistemas da Informação, da Área das Ciências Naturais, da Computação e das Engenharias, do Centro Universitário para o Desenvolvimento do Alto Vale do Itajaí, como condição parcial para a obtenção do grau de Bacharel em Sistemas de Informação.

Prof. Orientador: Ademar Perfoll Junior

RIO DO SUL
2026

CENTRO UNIVERSITÁRIO PARA O DESENVOLVIMENTO DO ALTO VALE DO ITAJAÍ - UNIDAVI

GUSTAVO BORGONHA

SISTEMA DE APOIO À ESTIMATIVA DE ESFORÇO EM DEMANDAS DE BUSINESS INTELLIGENCE BASEADO EM DADOS HISTÓRICOS E REGRESSÃO

Trabalho de Conclusão de Curso a ser apresentado ao curso de Sistemas da Informação, da Área das Ciências Naturais, da Computação e das Engenharias, do Centro Universitário para o Desenvolvimento do Alto Vale do Itajaí- UNIDAVI, a ser apreciado pela Banca Examinadora, formada por:

_____

Professor Orientador: Ademar Perfoll Junior

Banca Examinadora:

_____

Prof.

_____

Prof.

Rio do Sul, xx de mês de 2026.

Epígrafe, Epígrafe, Epígrafe, Epígrafe, Epígrafe, Epígrafe, Epígrafe, Epígrafe, Epígraıe, Epígrafe, Epígrafe, Epígrafe (Autor).

Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória Dedicatória.

AGRADECIMENTOS

Agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos agradecimentos.

RESUMO

Elemento obrigatório, que consiste na apresentação concisa dos pontos relevantes de um texto. O resumo deve dar uma visão rápida e clara do conteúdo e das conclusões do trabalho. Deve conter entre 150 a 500 palavras.

Palavras-Chave: palavra 1, palavra 2, palavra 3.

ABSTRACT

Versão do resumo em inglês (também pode ser em espanhol).

Keywords: keyword 1, keyword 2, keyword 3.

LISTA DE FIGURAS

Figura 1- Fluxograma do processo do desenvolvimento do sistema 33

LISTA DE QUADROS

Quadro 1 - Título Quadro 21

Quadro 2 - Título Quadro 21

Quadro 3 - Título Quadro 21

Quadro 4 - Título Quadro 21

Quadro 5 - Título Quadro 21

LISTA DE ABREVIATURAS E SIGLAS

APF	Análise de Pontos de Função
AutoML	*Automated Machine Learning*
API	*Application Programming Interface*
BI	*Business Intelligence*
COCOMO	*Constructive Cost Model*
HTML	*HyperText Markup Language*
HTTP	*Hypertext Transfer Protocol*
IA	Inteligência Artificial
JSON	*JavaScript Object Notation*
MAE	*Mean Absolute Error*
MdMRE	*Median Magnitude of Relative Error*
MMRE	*Maximal Marginal Relevance*
PHP	PHP*: Hypertext Preprocessor*
REST	*Representational State Transfer*
SQL	*Structured Query Language*
SVR	*Support Vector Regression*
TPOT	*Tree-based Pipeline Optimization Tool*
WWW	*World Wide Web*

SUMÁRIO

1. INTRODUÇÃO 13

1.1 PROBLEMA DE PESQUISA 14

1.2 OBJETIVOS 14

1.2.1 Geral 14

1.2.2 Específicos 14

1.3 JUSTIFICATIVA 15

2. REFERENCIAL TEÓRICO 16

2.1 ENGENHARIA DE SOFTWARE E GESTÃO DE PROJETOS 16

2.1.1 Ciclo de vida do software 16

2.1.2 Gestão de custos e prazos 18

2.1.3 O problema da subestimativa 19

2.2 ESTIMATIVA DE ESFORÇO EM SOFTWARE 20

2.2.1 Métodos baseados em especialistas 21

2.2.2 Métodos paramétricos e algorítmicos 21

2.2.3 Desafios na estimativa de software 22

2.3 INTELIGÊNCIA ARTIFICIAL E MACHINE LEARNING 23

2.3.1 Conceitos fundamentais 24

2.3.2 Aprendizado supervisionado 25

2.3.3 Algoritmos de regressão 26

2.3.4 Tratamento e análise de dados 27

2.4 TECNOLOGIAS APLICAS AO DESENVOLVIMENTO WEB 28

2.4.1 Arquitetura de aplicações web 29

2.4.2 Banco de dados relacionais 30

2.4.3 Backend e processamento de dados 30

2.4.4 Frontend e tipagem estatica 31

2.5 TRABALOS CORRELATOS 32
3. METODOLOGIA DE PESQUISA 34
4. RESULTADOS DO TC 36
5. CONCLUSÃO 37

1. INTRODUÇÃO

Ao longo da história da engenharia de software, a estimativa de prazos e custos sempre foi um dos mais complexos desafios da área, desde grandes empresas internacionais até para desenvolvedores autônomos (Brooks, 1995). Autores renomados da área deixam claro que a grande maioria das estimativas é feita na intuição e no julgamento, porém esse método, como citado por McConnell (2006), resulta muitas vezes em falhas preditivas, uma vez que os sistemas são mutáveis, pode haver um otimismo infundado e uma omissão de tarefas periféricas por parte de quem faz a estimativa, provando que "a experiência não tem sido sempre um bom indicador de resultados futuros" (Pressman; Maxim, 2016, p. 733). Essa subjetividade na estimativa compromete grandes cronogramas e ameaça a viabilidade de pequenas empresas de desenvolvimento, pois conforme afirma McConnell (2006), subestimar o esforço torna os planos inúteis e letais ao negócio.

Inserido nesse cenário de alta imprevisibilidade técnica, o desenvolvimento de soluções de dados e b*usiness intelligence* também é afetado por esses desafios. Na engenharia de software, a estimativa atua como o alicerce de todo o planejamento, consistindo na tentativa de determinar o esforço, os recursos, os custos e o tempo necessários para a construção de um sistema (Pressman; Maxim, 2016). A importância dessa prática não é apenas tentar prever o futuro exato, mas sim fornecer uma visão clara da realidade para determinar se as metas de negócio são realistas, permitindo que a liderança tenha controle sobre o projeto (McConnell, 2006). Contudo, depender de planejamentos estritamente manuais expõe o projeto a um risco muito maior de falhas preditivas, o que pode resultar em prejuízos financeiros alarmantes e na quebra de confiança com o cliente (Pressman; Maxim, 2016).

Para dimensionar o impacto dessas falhas de estimativa, um estudo da consultoria McKinsey (Bloch; Blumberg; Laartz, 2012) avaliou mais de 5.400 grandes projetos de TI e revelou que os estouros de orçamento geraram uma perda acumulada de 66 bilhões de dólares, com projetos de software excedendo, em média, 66% de seus custos planejados. Esse desperdício de capital é um problema histórico do setor, corroborado pelo *Chaos Report* (The Standish Group, 1995), que estimou que, em um único ano, corporações e governos norte-americanos desperdiçaram 81 bilhões de dólares em projetos de software cancelados e pagaram 59 bilhões adicionais por projetos que excederam o tempo original. Assim, surge o interesse no mercado e no meio científico em realizar a transição dessas abordagens informais para métodos fundamentados em métricas quantitativas e análises empíricas de dados. Para mitigar essa dor

da incerteza e otimizar a orçamentação técnica, o presente trabalho desenvolverá um sistema preditivo dedicado a apoiar a estimativa de esforço em demandas de b*usiness intelligence* e extração de dados.

Esta aplicação utilizará inteligência artificial, especificamente por meio de algoritmos de m*achine learning* para regressão, com o objetivo de substituir a estimativa humana por predições baseadas no histórico de horas de demandas passadas. A infraestrutura tecnológica da solução contará com uma arquitetura segmentada e moderna, englobando o processamento de dados e regras de negócio via Python e a biblioteca Pandas no *backend*, a persistência segura e estruturada das informações através do banco de dados relacional PostgreSQL, e uma interface de gestão reativa e dinâmica construída com o *framework* Angular. Dessa forma, o sistema buscará fornecer previsões quantitativas exatas, conferindo maior segurança na tomada de decisão financeira.

Além deste capítulo introdutório, o trabalho apresenta uma segunda parte onde serão abordadas as tecnologias usadas no desenvolvimento da aplicação. No terceiro capítulo, será descrita a metodologia de pesquisa. O quarto capítulo apresentará o desenvolvimento e o andamento do projeto, e o último capítulo reunirá as conclusões e as considerações finais.

1.1 PROBLEMA DE PESQUISA

Como modelos preditivos baseados em dados históricos podem contribuir para a precisão da estimativa de esforço em demandas de bu*siness intelligence* desenvolvidas por pequenas empresas e profissionais independentes?

1.2 OBJETIVOS

1.2.1 Geral

- Desenvolver um sistema web preditivo baseado em algoritmos de regressão e dados históricos para auxiliar na tomada de decisão e no controle de orçamentos de demandas de b*usiness intelligence*.

1.2.2 Específicos

- Identificar dados históricos relacionados a demanda da área de dados;
- Estruturar os dados históricos em um formato padronizado;
- Mapear variáveis relevantes para a estimativa de esforço;

- Implementar modelos de regressão para estimar o esforço das demandas;
- Comparar o desempenho dos modelos treinados utilizando métricas quantitativas de erro;
- Elaborar uma interface web para cadastro, consulta e apoio à estimativa de novas demandas;
- Avaliar se o modelo pode apoiar o processo de orçamentação em relação às estimativas feitas manualmente.

1.3 JUSTIFICATIVA

A estimativa imprecisa de tempo e custos é um desafio histórico na engenharia de software, resultando frequentemente em atrasos, orçamentos estourados e prejuízos. Quando esse cenário é transposto especificamente para o contexto de demandas de *Business Intelligence* e processos de dados, essa dificuldade mostra-se ainda mais crítica para desenvolvedores autônomos e pequenas empresas de tecnologia, pois erros de precificação ao planejar o esforço técnico de integrações e criação de painéis gerenciais podem comprometer a viabilidade financeira do projeto e a relação de confiança com o cliente.

Vivenciada cotidianamente na prática profissional do autor com produtos de BI, observa-se que a orçamentação manual de novas demandas frequentemente esbarra em variáveis operacionais imprevisíveis. Em um cenário típico, por exemplo, o esforço para criar um painel costuma ser subestimado devido a tarefas periféricas que surgem apenas na execução, como a limpeza de fontes não padronizadas, instabilidade de conexões ou mudanças de escopo pelos usuários.

Nesse contexto, o estudo deste problema justifica-se pela necessidade técnica e comercial de mitigar a subjetividade humana que historicamente contamina as estimativas de software. Ao propor a substituição de cálculos informais por um modelo preditivo baseado em *Machine Learning*, este trabalho ataca a causa das falhas de orçamentação. Além disso, a relevância acadêmica reside na investigação prática desse método, permitindo mapear quais variáveis de BI exercem o maior impacto no tempo, substituindo palpites por evidências rigorosas.

2. REFERENCIAL TEÓRICO

Neste capítulo, serão abordados os conceitos, ferramentas e linguagens empregados no desenvolvimento do projeto.

2.1 ENGENHARIA DE SOFTWARE E GESTÃO DE PROJETOS

A crescente complexidade das soluções computacionais e a dependência tecnológica do mercado atual tornaram o desenvolvimento de sistemas uma atividade que não pode mais ser baseada apenas na intuição e no esforço não estruturado. Para trazer ordem a esse ambiente, Pressman e Maxim (2016) definem a engenharia de software como a aplicação de uma abordagem sistemática, disciplinada e quantificável no desenvolvimento, na operação e na manutenção de aplicações. A adoção dessa disciplina engloba o uso de processos, métodos e ferramentas projetados especificamente para possibilitar a construção de sistemas complexos com alta qualidade e, primordialmente, dentro dos prazos estabelecidos.

No entanto, para que essa base puramente técnica produza os resultados esperados de forma sustentável, ela necessita estar intrinsicamente aliada a uma sólida gestão de projetos. Sob essa perspectiva, Pressman e Maxim (2016) conceituam o gerenciamento de projetos de software como a camada administrativa e de controle que envolve o planejamento, a monitoração e a organização das pessoas, dos processos e dos eventos, acompanhando a aplicação desde a sua concepção preliminar até a sua disponibilização operacional ao usuário. É exatamente na intersecção entre a execução técnica (engenharia) e a organização estratégica (gestão) que as equipe conseguem dominar a imprevisibilidade, definir cronogramas, estabelecer orçamentos viáveis e garantir que a solução final entregue o real valor de negócio exigido pelo cliente.

2.1.1 Ciclo de vida do software

O ciclo de vida do software atua como uma estrutura adaptável, guiando e organizando as atividades necessárias para a criação do sistema. De acordo com Pressman e Maxim (2016), a engenharia de software engloba processos, métodos e ferramentas essenciais para construir sistemas de software complexos com qualidade e dentro do prazo. A adoção desse roteiro evita o caos e estabelece cinco atividades metodológicas genéricas: comunicação, planejamento, modelagem, construção e entrega.

É na fase de planejamento, logo após o entendimento inicial do problema, que a orçamentação se encaixa de forma decisiva. O ato de estimar atua como o alicerce gerencial do ciclo de vida, sendo uma predição essencial para viabilizar o negócio. No rigor da palavra, McConnell (2006) estabelece que uma estimativa consiste fundamentalmente na previsão do tempo de duração ou do custo financeiro exigido por um projeto. Conforme o mesmo autor complementa, para ser considerada eficaz, essa estimativa deve fornecer uma visão suficientemente clara da realidade, permitindo que a liderança tenha o controle do projeto e tome decisões seguras.

Contudo, orçar com exatidão é um desafio crônico, especialmente para desenvolvedores autônomos e pequenas empresas de tecnologia. Nesses contextos, erros na precificação comprometem a viabilidade financeira e destroem a confiança do cliente. A literatura alerta sobre o perigo de se criar orçamentos sem bases quantitativas sólidas. Nesse sentido, Brooks (1995) adverte que estimativas elaboradas sem o rigor de um método numérico, apoiadas em poucas evidências e motivadas sobretudo por pressões gerenciais, geram cenários altamente arriscados e impossíveis de serem sustentados tecnicamente.

Essa dificuldade ocorre devido à inerente incerteza das fases iniciais do desenvolvimento. Sobre essa imprevisibilidade e a necessidade de usar o histórico para embasar as predições, Pressman e Maxim destacam:

> O planejamento exige que você assuma um compromisso inicial, mesmo que mais tarde ele venha se mostrar errado. Sempre que forem feitas estimativas, olhamos para o futuro e aceitamos certo grau de incerteza. [...] As métricas de projeto e processo podem proporcionar perspectivas históricas e informações valiosas para gerar estimativas quantitativas (Pressman; Maxim, 2016, p. 728-729).

Dessa forma, conclui-se que as abordagens baseadas puramente no julgamento empírico manual falham em fornecer o rigor técnico necessário para a sobrevivência mercadológica (McConnell, 2006). Para preencher essa lacuna no planejamento, a engenharia de software defende a adoção de modelos empíricos de estimativa fundamentados em métricas de processo e perspectivas históricas de projetos anteriores. Conforme corroboram Pressman e Maxim (2016), essa transição metodológica permite substituir cálculos estáticos e subjetivos por avaliações quantitativas baseadas em dados reais do ciclo de vida do desenvolvimento, garantindo orçamentos muito mais rigorosos, controláveis e alinhados à verdadeira complexidade do sistema a ser construído.

2.1.2 Gestão de custos e prazos

A gestão de projetos transcende a simples codificação, ela atua como a camada administrativa que envolve o planejamento, o monitoramento e o controle contínuo de pessoas, processos e eventos à medida que o software evolui (Pressman; Maxim, 2016). Nesse cenário, o controle de custos e prazos desponta como o pilar financeiro que sustenta a aplicação comercial. Para que essa gestão seja efetiva, é fundamental compreender a verdadeira função de um orçamento preliminar. De acordo com McConnell (2006), o objetivo primário da estimativa não é apenas tentar prever o futuro exato do projeto, mas sim fornecer uma visão clara da realidade para determinar se as metas de negócio são realistas o suficiente para permitir que a liderança tome boas decisões e atinja seus alvos.

Essa necessidade de ancorar as metas na realidade técnica - que, de acordo com McConnell (2006), é o verdadeiro propósito de uma estimativa para avaliar se os objetivos são realistas - é ainda mais crítica no contexto de pequenas empresas de tecnologia e desenvolvedores autônomos. Diferentemente de grandes corporações, que frequentemente possuem margens financeiras robustas para absorver os prejuízos de sistemas mal calculados, o profissional autônomo opera com recursos estritamente limitados. Para esse perfil, a estimativa imprecisa de tempo e custos ultrapassa a barreira do mero incômodo no cronograma, configurando-se como um fator de risco letal para a sustentabilidade da operação, uma vez que as penalidades financeiras e operacionais por subestimar o esforço de um projeto são severas (McConnell, 2006) e ameaçam diretamente a viabilidade comercial do negócio (Pressman; Maxim, 2016).

A literatura técnica adverte reiteradamente sobre esse perigo inerente ao planejamento, destacando que as penalidades geradas por subestimar o esforço de um projeto de software (como a redução da eficácia da equipe) são severamente mais graves do que os problemas decorrentes de uma superestimativa (McConnell, 2006). Em um cenário de escassez, erros na precificação inicial resultam em orçamentos estourados que destroem a viabilidade do próprio negócio e corroem a relação de confiança estabelecida com o cliente final. Fica evidente, portanto, que a orçamentação precisa evoluir de um simples palpite para uma prática mais segura e rigorosa. Contudo, para que essa evolução aconteça, é essencial compreender os fatores humanos e técnicos que historicamente tornam a precisão preditiva um desafio tão complexo no desenvolvimento de software.

2.1.3 O problema da subestimativa

Apesar da notória importância de um planejamento financeiro rigoroso, a indústria de software sofre historicamente com a dificuldade crônica de estimar prazos. Esse problema estrutural, conhecido como subestimativa, ocorre porque a criação de sistemas é um processo iterativo e altamente suscetível a variáveis humanas e tecnológicas. Avaliando as raízes desse problema, McConnell (2006) aponta que as principais fontes de erro nas estimativas derivam do esquecimento de atividades periféricas e de um otimismo infundado por parte da equipe. Ou seja, os desenvolvedores tendem, por natureza, a basear suas predições sempre no melhor cenário possível, negligenciando os percalços naturais da engenharia de software.

Além dessa subjetividade inicial, os próprios profissionais da área técnica frequentemente subestimam a real complexidade do esforço de codificação. Para ilustrar essa limitação preditiva humana no planejamento, Pressman e Maxim (2016) citam a irreverente regra 90-90, na qual os primeiros 90% de um sistema absorvem 90% dos esforços e tempos alocados, enquanto os 10% restantes consomem outros 90% de esforço e tempo. Essa profunda distorção matemática ocorre porque as etapas finais de integração e testes costumam expor os riscos técnicos que não puderam ser previstos no início, estourando completamente o orçamento original.

Sobre a forma silenciosa como essas falhas acabam destruindo o cronograma, Pressman e Maxim (2016) relatam que, ao ser questionado sobre como os grandes projetos de software se atrasam, o renomado engenheiro Fred Brooks respondeu com uma constatação tão simples quanto profunda: "Um dia de cada vez". Essa dificuldade em perceber o acúmulo de pequenos atrasos no cotidiano é frequentemente agravada por falsas suposições gerenciais, como a ideia de que adicionar mais mão de obra resolverá o problema. Conhecida na literatura como a Lei de Brooks, a regra alerta que a inclusão de novos profissionais em um cronograma já comprometido tende a agravar ainda mais o atraso do projeto (Pressman; Maxim, 2016) visto que o tempo gasto instruindo os novos membros reduz drasticamente a produtividade de quem já está focado no trabalho.

Dessa forma, fica evidente que orçar projetos de tecnologia puramente com base na intuição ou no julgamento empírico isolado é uma abordagem metodologicamente falha. Conforme alerta McConnell (2006), as principais fontes de erro nessas avaliações manuais derivam justamente de um otimismo infundado por parte dos desenvolvedores e da omissão

sistemática de tarefas periféricas que permanecem invisíveis durante o planejamento prematuro. Aliados às surpresas estruturais do código, esses fatores criam um cenário de severa imprecisão que atua como um verdadeiro risco de negócio, ameaçando diretamente a sustentabilidade e a sobrevivência financeira das empresas (Pressman; Maxim, 2016). Para combater essa subjetividade nociva e proteger a operação, torna-se imperativa a transição dos palpites manuais para os métodos empíricos e quantitativos de estimativa de esforço. Afinal, como defende Brooks (1995), sustentar uma estimativa de maneira firme e plausível torna-se uma tarefa extremamente difícil quando esta não é fundamentada em métodos quantitativos. É exatamente o estudo detalhado desses métodos preditivos — englobando desde as abordagens empíricas tradicionais até a evolução algorítmica orientada a dados — que servirá de base para a próxima seção.

2.2 ESTIMATIVA DE ESFORÇO EM SOFTWARE

A estimativa de esforço é uma etapa gerencial crítica, mas a prática atual do mercado reflete um forte distanciamento entre a teoria acadêmica e a rotina dos desenvolvedores. Na teoria, a engenharia de software recomenda que o ato de orçar utilize técnicas de decomposição, que assumem uma estratégia do tipo dividir para conquistar, separando o projeto em funções menores que podem ser estimadas de maneira estruturada (Pressman; Maxim, 2016). No entanto, a realidade técnica demonstra que a maioria esmagadora das estimativas ainda foge desse rigor. Como aponta a pesquisa citada por McConnell (2006), abordagens baseadas puramente na intuição, no palpite e no julgamento empírico desestruturado (como a lembrança informal de projetos anteriores) representam as estratégias dominantes, sendo utilizadas na elaboração de 60% a 85% de todos os orçamentos de software.

Esses métodos baseados puramente no julgamento humano falham sistematicamente porque a construção de sistemas é complexa e a nossa mente é suscetível ao otimismo infundado, o que faz com que desenvolvedores tendam a gerar cronogramas sempre muito otimistas (McConnell, 2006). A literatura alerta que o uso exclusivo de palpites e memória está diretamente correlacionado a projetos cancelados ou estourados. Sobre o perigo de se orçar com base apenas no instinto de quem vai programar, Pressman e Maxim (2016) ressaltam que a experiência não é um bom indicador de projetos posteriores. Os autores advertem que qualquer técnica viável precisa estar ancorada no passado, pois, se não existem dados históricos, os custos se baseiam em uma fundação muito precária. Logo, a falha dos métodos atuais evidencia

a urgência de se abandonar as estimativas baseadas em opiniões e adotar ferramentas preditivas orientadas por dados.

2.2.1 Métodos baseados em especialistas

Historicamente, a abordagem mais comum na indústria para definir prazos e custos é o julgamento de especialistas, que fundamenta entre 72% e 83% de todas as estimativas criadas (McConnell, 2006). Esse método baseia-se puramente na intuição e na memória do desenvolvedor. Para tentar reduzir os erros extremos dessa avaliação individual, o mercado popularizou técnicas de estimativa em grupo, como o *Planning Poker*. Essa dinâmica ágil é uma adaptação do método *Wideband Delphi*, no qual os profissionais discutem o escopo e votam iterativamente para chegar a um consenso, seguindo a mesma lógica colaborativa do "jogo de planejamento" introduzido pela metodologia *Extreme Programming* (Pressman; Maxim, 2016).

Apesar de atenuarem falhas individuais, essas estimativas em grupo não eliminam o principal calcanhar de aquiles da categoria: a profunda subjetividade humana. McConnell (2006) alerta que a intuição técnica é frequentemente contaminada por vieses inconscientes e por um otimismo infundado que ignora riscos e tarefas periféricas. Além das limitações cognitivas, há o fator político: quando pressionados comercialmente, os desenvolvedores tendem a ceder e reduzir as suas estimativas na intuição, sem qualquer embasamento técnico que justifique o novo prazo (McConnell, 2006). Portanto, depender exclusivamente do julgamento humano gera um grau inaceitável de imprevisibilidade para a sobrevivência de desenvolvedores autônomos, evidenciando o colapso das técnicas empíricas e a necessidade de métodos orientados a dados.

2.2.2 Métodos paramétricos e algorítmicos

Historicamente, para mitigar a falha e a subjetividade do julgamento humano, a engenharia de software criou modelos empíricos que utilizam fórmulas matemáticas e dados históricos quantitativos para estimar o esforço de um projeto. Destaca-se nessa vertente a Análise de Pontos de Função (APF) que, conforme definem Pressman e Maxim (2016), é uma métrica que mede o tamanho do software com base puramente na funcionalidade entregue ao usuário (como o número de entradas, saídas e consultas), sendo uma medida independente da linguagem de programação que será adotada. Outro modelo matemático clássico é o COCOMO

(*Constructive Cost Model*), desenvolvido originalmente por Barry Boehm, que utiliza a estimativa do tamanho do sistema combinada a fatores de ajuste de complexidade para prever o esforço e os custos de desenvolvimento.

Contudo, a aplicação rigorosa desses modelos tradicionais exige uma sobrecarga burocrática que os torna inviáveis para desenvolvedores autônomos e pequenas empresas de tecnologia. A validade dessas fórmulas matemáticas depende de especificações de requisitos extensas e congeladas desde o primeiro dia do projeto, além de exigir que o analista calcule manualmente diversos fatores de ajuste — o que, de acordo com McConnell (2006), na prática acaba apenas introduzindo novas chances para o erro humano se infiltrar na estimativa. Esse engessamento prematuro do escopo contrasta violentamente com a realidade mutável do desenvolvimento moderno, onde é frequentemente impossível prever como o escopo de um sistema irá evoluir, exigindo flexibilidade contínua da equipe para responder a um ambiente de negócios altamente fluido (Pressman; Maxim, 2016).

Tudo isso evidencia a necessidade de substituir esses métodos clássicos burocráticos por soluções preditivas mais dinâmicas e automatizadas pela inteligência artificial. Essa transição tecnológica é vital pois, conforme fundamentam Raschka e Mirjalili (2019), em vez de exigir que os desenvolvedores deduzam regras de orçamentação e construam modelos estáticos manualmente, o uso do m*achine learning* oferece uma alternativa algorítmica muito mais eficiente. A inteligência artificial permite que o sistema aprenda capturando padrões ocultos diretamente de bases de dados do passado, possibilitando a tomada de decisões e a geração de predições com alto grau de exatidão.

2.2.3 Desafios na estimativa de software

Nos últimos anos, a indústria vivenciou uma forte transição para os métodos ágeis, buscando lidar com requisitos voláteis e ambientes de negócios em constante mudança. No entanto, essa flexibilidade trouxe um novo dilema gerencial: como estimar custos exatos quando o próprio escopo do sistema é mutável? A dificuldade estrutural de prever horas em tarefas complexas é ilustrada por McConnell (2006) através do conceito do "Cone da Incerteza". O autor demonstra que estimativas criadas na fase de concepção inicial do projeto sofrem com um altíssimo grau de indefinição, podendo ser imprecisas por um fator de até quatro vezes (para mais ou para menos). Essa constatação prova que o erro preditivo inicial não é apenas uma

incompetência do desenvolvedor, mas sim uma consequência da incerteza inerente ao próprio software antes que as principais decisões de projeto sejam tomadas.

Para contornar essa incerteza, a cultura ágil sugere que os orçamentos sejam fatiados e reavaliados a cada novo ciclo iterativo. Contudo, essa abordagem colide com a realidade comercial dos desenvolvedores autônomos e das pequenas empresas. Conforme aponta McConnell (2006), no mundo real os clientes valorizam fortemente a "previsibilidade" - a garantia de saber custos, prazos e funcionalidades antecipadamente -, preferindo-a muitas vezes em detrimento da própria flexibilidade de alterar os requisitos. O profissional autônomo fica, portanto, encurralado entre a enorme incerteza técnica do início do projeto e a exigência contratual do cliente por um preço fechado. Como a intuição humana não consegue processar essa imprevisibilidade com segurança, torna-se imperativo adotar modelos quantitativos ancorados firmemente em dados do passado, o que pavimenta e justifica a introdução de algoritmos preditivos e inteligência artificial na orçamentação.

2.3 INTELIGÊNCIA ARTIFICIAL E MACHINE LEARNING

Diante das limitações cognitivas e da subjetividade inerentes aos métodos tradicionais de orçamentação, a adoção de abordagens baseadas em dados surge como uma evolução indispensável para a engenharia de software. É nesse cenário que a inteligência artificial (IA) fundamenta a solução para o problema preditivo. De forma abrangente, a IA é um campo focado não apenas na compreensão do pensamento, mas na construção de entidades e sistemas capazes de perceber, raciocinar e computar ações de maneira eficaz e segura diante de situações complexas (Russell; Norvig, 2021). Dentro desse vasto ecossistema tecnológico, o aprendizado de máquina (m*achine learning*) atua como a subárea prática que confere aos computadores a habilidade essencial de melhorar o seu desempenho com base na experiência acumulada em conjuntos de dados.

A aplicação do m*achine learning* altera fundamentalmente o paradigma de resolução de problemas computacionais. De acordo com Raschka e Mirjalili (2019), em vez de exigir que humanos derivem regras e construam modelos estáticos manualmente a partir da análise dos dados, essa tecnologia oferece uma alternativa muito mais eficiente: o uso de algoritmos de autoaprendizado para capturar o conhecimento oculto, melhorando gradualmente o desempenho de modelos preditivos para a tomada de decisões baseada em dados. Para o desafio da estimativa de software, isso significa que é possível treinar um modelo com o histórico de projetos passados para prever o esforço de codificação futuro, permitindo que desenvolvedores

autônomos e pequenas empresas abandonem a intuição humana em prol de orçamentos guiados por rigorosos cálculos matemáticos.

2.3.1 Conceitos fundamentais

A inteligência artificial é um campo abrangente da ciência da computação dedicado não apenas à compreensão da inteligência, mas, sobretudo, à construção de entidades e sistemas capazes de raciocinar e agir de forma eficaz e segura em situações complexas e inéditas (Russell; Norvig, 2021). Ao longo de sua evolução histórica, a IA foi abordada sob diferentes perspectivas, variando desde a tentativa empírica de imitar fielmente o pensamento e comportamento humano até abordagens matemáticas focadas puramente na racionalidade lógica. Atualmente, o modelo padrão e predominante na literatura define a IA como o estudo e a construção de agentes racionais, ou seja, sistemas que recebem percepções do ambiente e executam ações buscando atingir o melhor resultado esperado possível (maximizar a utilidade) para um determinado objetivo (Russell; Norvig, 2021).

Dentro desse vasto universo da inteligência artificial, o Aprendizado de Máquina (m*achine learning*) estabeleceu-se como uma de suas subáreas mais vitais. Enquanto a IA engloba qualquer técnica que permita a um computador imitar o comportamento inteligente, o m*achine learning* é especificamente definido como o estudo da capacidade de um sistema melhorar o seu desempenho com base na experiência. De acordo com Raschka e Mirjalili (2019), o m*achine learning* evoluiu na segunda metade do século XX como um campo que utiliza algoritmos de autoaprendizado para derivar conhecimento diretamente dos dados. Em vez de exigir que desenvolvedores deduzam regras e construam modelos lógicos manualmente, essa tecnologia oferece uma alternativa muito mais eficiente para capturar padrões ocultos e tomar decisões orientadas por dados.

A principal diferença entre os conceitos reside, portanto, na relação de subordinação e no método: a IA é o objetivo maior (criar uma máquina inteligente), enquanto o m*achine learning* é um dos caminhos técnicos pelo qual esse objetivo é alcançado sem a necessidade de programação explícita de regras (Raschka; Mirjalili, 2019). No contexto da estimativa de projetos de software, essa distinção é fundamental. Uma abordagem tradicional de IA tentaria criar um sistema especialista baseado em centenas de regras rígidas inseridas por humanos

sobre como orçar um projeto — o que a literatura já demonstrou ser falho, uma vez que tais métodos engessados quebram frente à incerteza inerente à programação e à incapacidade do sistema de aprender com a experiência (Russel; Norvig, 2021). O uso do m*achine learning*, por outro lado, elimina a necessidade de construir essas regras manuais, alimentando o sistema diretamente com o histórico de projetos passados para que o próprio algoritmo encontre as relações matemáticas entre as variáveis do escopo e o tempo de desenvolvimento, viabilizando predições de esforço muito mais eficientes e precisas (Raschka; Mirjalili, 2019).

2.3.2 Aprendizado supervisionado

O aprendizado supervisionado (s*upervised learning*) é uma importante vertente da inteligência artificial para solucionar problemas preditivos. Segundo James *et al.* (2013), esse método consiste na construção de um modelo estatístico para prever uma saída com base em variáveis de entrada. O termo supervisionado indica o uso de dados de treinamento onde os sinais de saída (rótulos ou *labels*) já são previamente conhecidos (Raschka; Mirjalili, 2019). O objetivo algorítmico é extrair conhecimento estruturado desses dados rotulados para encontrar uma função estimada $\hat{f}$ que se aproxime ao máximo da relação matemática real subjacente (James *et al.*, 2013).

Dentro dessa categoria, os problemas são divididos em duas grandes vertentes operacionais: classificação e regressão. A classificação é utilizada para prever rótulos qualitativos e categóricos (James *et al.*, 2013), enquanto a análise de regressão é aplicada quando a variável de resposta possui um valor contínuo (Raschka; Mirjalili, 2019). Consequentemente, em contextos em que o objetivo analítico é prever uma quantidade numérica exata — como avaliações contínuas de tempo, esforço ou custos —, a modelagem recai intrinsecamente sobre as técnicas de regressão. Nesses modelos, o algoritmo ajusta seus parâmetros matemáticos para prever a variável contínua alvo com base estrita nas observações anteriores.

O fluxo de trabalho típico desse aprendizado envolve a extração de dados rotulados, que são passados ao algoritmo para o treinamento e a consequente geração de um modelo preditivo maduro (Raschka; Mirjalili, 2019). Uma vez treinado, o sistema adquire a capacidade de receber novos dados de entrada ainda não rotulados e retornar uma predição fundamentada sobre o futuro. É exatamente essa capacidade baseada em dados empíricos que torna o aprendizado supervisionado superior aos métodos baseados exclusivamente no julgamento manual. Afinal,

em vez de exigir que humanos deduzam regras e construam modelos estáticos manualmente a partir da análise de grandes volumes de informações, o uso do *machine learning* oferece uma alternativa automatizada e muito mais eficiente para capturar o conhecimento, substituindo a subjetividade humana por predições guiadas exclusivamente por dados (Raschka; Mirjalili, 2019).

2.3.3 Algoritmos de regressão

No contexto do aprendizado supervisionado, quando a variável de saída que se deseja prever assume valores numéricos contínuos — como a quantidade exata de horas necessárias para desenvolver um novo projeto de software —, o problema é classificado essencialmente como uma tarefa de regressão (James *et al.*, 2013; Raschka; Mirjalili, 2019). O modelo analítico fundamental para essa categoria é a Regressão Linear, que busca estabelecer matematicamente a relação entre as variáveis explicativas de entrada (como o escopo ou número de integrações) e a variável alvo contínua, encontrando a reta ou hiperplano que melhor se ajusta aos dados de treinamento para minimizar as distâncias dos erros residuais (Raschka; Mirjalili, 2019). Contudo, embora a regressão linear tradicional seja altamente interpretável e eficiente em cenários onde a relação de causa e efeito é estritamente proporcional, a engenharia de software frequentemente lida com relacionamentos não lineares complexos, exigindo abordagens algorítmicas mais flexíveis para garantir a precisão da predição.

Para superar as restrições dos modelos puramente lineares, as árvores de decisão para regressão (r*egression trees*) despontam como uma alternativa poderosa e dinâmica. De acordo com James *et al.* (2013), esse método envolve estratificar ou segmentar o espaço dos preditores em um número de regiões distintas e não sobrepostas, utilizando regras lógicas de divisão baseadas nas características dos dados. Em vez de calcular uma equação linear global única, a árvore de decisão para regressão subdivide o espaço de entrada complexo em regiões menores e mais gerenciáveis, prevendo para uma nova observação a média exata dos valores contínuos de treinamento que pertencem àquela mesma região (Raschka; Mirjalili, 2019). Essa característica torna o algoritmo excelente para lidar com variáveis técnicas categóricas e dados não lineares sem a necessidade de transformações matemáticas exaustivas (Raschka; Mirjalili, 2019). Entretanto, a literatura adverte que as árvores de decisão individuais sofrem com um problema estrutural: a alta variância e a forte tendência ao sobre ajuste (*overfitting*), o que

significa que pequenas mudanças nos dados de treinamento podem gerar representações e orçamentos completamente diferentes (James *et al.*, 2013).

Para mitigar a instabilidade matemática e a alta variância das árvores individuais, a ciência de dados recomenda o uso de métodos de *ensemble* (conjunto), sendo o *Random Forest* (Floresta Aleatória) a técnica de regressão com maior destaque para essa finalidade. Como explicam Raschka e Mirjalili (2019), o *Random Forest* é uma técnica de conjunto que constrói múltiplas árvores de decisão profundas a partir de amostras de dados aleatórias. O resultado preditivo final da variável contínua é obtido por meio da média das previsões de todas as árvores individuais geradas no modelo, o que reduz drasticamente a variância global e o torna muito mais robusto contra ruídos e outliers do que uma árvore operando isoladamente (Raschka; Mirjalili, 2019). No desafio da orçamentação de software, a aplicação de algoritmos de regressão em conjunto, como o *Random Forest*, permite analisar um vasto histórico de dados para fornecer estimativas de tempo com uma confiabilidade preditiva muito superior aos palpites humanos. Essa estabilidade técnica é viável pois, de acordo com Raschka e Mirjalili (2019), a combinação de múltiplas árvores de decisão diminui a variância do modelo preditivo, tornando-o estruturalmente mais robusto e menos sensível a eventuais valores atípicos (*outliers*) presentes na base de dados.

2.3.4 Tratamento e análise de dados

Conforme definem Raschka e Mirjalili (2019), a qualidade dos dados e a quantidade de informações úteis que eles contêm são os fatores fundamentais que determinam o quão bem um algoritmo de aprendizado de máquina pode aprender. Na prática da ciência de dados, a maior parte do esforço de desenvolvimento não é gasta no treinamento algorítmico, visto que as tarefas de preparação, limpeza, transformação e rearranjo de dados frequentemente chegam a consumir 80% ou mais do tempo de um analista (McKinney, 2022). No contexto do aprendizado de máquina, conjuntos de dados do mundo real frequentemente contêm inconsistências, exigindo que se examine e pré-processe rigorosamente as informações antes de alimentá-las em qualquer algoritmo ou modelo preditivo (Raschka; Mirjalili, 2019).

O primeiro desafio no tratamento de dados reais é lidar com valores ausentes (*missing data*), que ocorrem rotineiramente e podem gerar resultados imprevisíveis se forem simplesmente ignorados pela máquina, conforme apontam Raschka e Mirjalili (2019). Embora a exclusão direta de linhas com dados faltantes seja uma abordagem nativa, ela corre o risco de

remover muitas amostras e perder informações valiosas que o classificador precisa para discriminar os padrões de tempo e custo (Raschka; Mirjalili, 2019). Para contornar isso, a literatura recomenda o uso de técnicas de imputação, preenchendo as lacunas nos dados utilizando estatísticas como a média ou a mediana da característica em questão (Raschka; Mirjalili, 2019). Além desse tratamento, muitos algoritmos exigem o redimensionamento de recursos (como a padronização) para operarem de forma otimizada. No entanto, em arquiteturas preditivas baseadas em algoritmos como árvores de decisão e *Random Forest*, o modelo é beneficiado estruturalmente, pois esses algoritmos são invariantes à escala, isentando a necessidade de normalização prévia das variáveis numéricas (Raschka; Mirjalili, 2019).

O principal obstáculo metodológico na preparação dos dados reside na conversão de variáveis técnicas categóricas, uma vez que o histórico de projetos de software geralmente contém rótulos textuais, exemplo pode ser complexidade da funcionalidade ou linguagem de programação. Conforme definem Raschka e Mirjalili (2019), o tratamento divide-se entre as características ordinais, que podem ser compreendidas como valores categóricos que possuem uma ordem implícita, e as características nominais, que não implicam qualquer hierarquia dimensional. Para variáveis ordinais, como níveis de complexidade "Baixa, Média e Alta", aplica-se um mapeamento simples que as converte em sequências lógicas de números inteiros.

Por outro lado, lidar com variáveis nominais exige cautela. Atribuir valores inteiros crescentes (como 1, 2 e 3) para tecnologias distintas como "Python, Java ou PHP" induziria o algoritmo a assumir erroneamente que uma linguagem é matematicamente maior ou superior à outra, gerando predições subótimas. Para solucionar esse problema com variáveis nominais, aplica-se a técnica de *one-hot encoding,* também conhecida na estatística como criação de variáveis *dummy*. A ideia por trás dessa abordagem é criar uma característica fictícia — uma coluna binária independente contendo 0 ou 1 — para cada valor único na coluna de característica nominal. A aplicação coordenada dessas técnicas de limpeza e de codificação categórica, frequentemente operacionalizadas em Python por meio da biblioteca pandas, garante que o histórico de projetos seja convertido em uma matriz perfeitamente padronizada. É esse rigoroso pré-processamento que permitirá ao algoritmo preditivo focar exclusivamente em descobrir as relações ocultas entre o escopo técnico do sistema e o esforço real de desenvolvimento.

2.4 TECNOLOGIAS APLICADAS AO DESENVOLVIMENTO WEB

A construção de sistemas preditivos modernos na web exige uma arquitetura baseada na separação clara entre a interface (*frontend*), o servidor lógico (*backend*) e a camada de persistência de dados (Elmasri; Navathe, 2011). Essa segregação estrutural é amplamente recomendada na engenharia de software para garantir a independência, a escalabilidade e a manutenção isolada de cada módulo da aplicação (Martin, 2020). Dentro desse modelo arquitetural, os sistemas de bancos de dados relacionais — a exemplo do PostgreSQL — desempenham um papel fundamental. Conforme elucidam Elmasri e Navathe (2011), as soluções relacionais asseguram que o conjunto de dados operacionais, vital para o treinamento algorítmico contínuo, seja armazenado de forma estruturada, íntegra e com alta disponibilidade para as consultas do servidor central.

Em ecossistemas web preditivos, a inteligência e as regras de negócio operam na camada de *backend*, contexto em que o Python se destaca na construção de APIs (Ramalho, 2022). Devido à sua sintaxe concisa e ao seu ecossistema robusto (Ramalho, 2022), servidores desenvolvidos nessa linguagem, potencializados por bibliotecas analíticas como o Pandas (McKinney, 2022), atuam como pontes eficientes para integrar requisições HTTP diretamente a motores de *machine learning*. Por fim, a camada de *frontend* exige tecnologias capazes de gerar interatividade ágil, como o *framework* Angular (Freeman, 2017), frequentemente aliado à linguagem TypeScript para adicionar uma tipagem estática e segura ao desenvolvimento (Cherny, 2019). Essa combinação tecnológica no lado do cliente viabiliza a criação de interfaces dinâmicas (Freeman, 2017), fundamentais para que os usuários insiram parâmetros técnicos complexos e consumam resultados preditivos com total confiabilidade (Cherny, 2019).

2.4.1 Arquitetura de aplicações web

O desenvolvimento de aplicações web robustas exige uma arquitetura estruturada na separação estrita de responsabilidades para garantir escalabilidade e manutenção simplificada (Martin, 2020). Para atingir esse objetivo, recomenda-se a divisão dos sistemas em três camadas principais e isoladas: interface do usuário (*frontend*), servidor lógico (*backend*) e armazenamento de dados (Elmasri; Navathe, 2011). Conforme propõe Martin (2020), essa segregação é um princípio fundamental que isola as regras de negócio das tecnologias de apresentação, garantindo que as interfaces evoluam de forma independente da inteligência central. Nesse modelo, a camada de front-end atua como a interface interativa na qual os

usuários inserem os parâmetros de entrada, comunicando-se de forma assíncrona com o servidor por meio do envio e recebimento de requisições HTTP estruturadas no formato JSON.

O núcleo orquestrador e analítico de sistemas reside na camada de *backend* (Martin, 2020). Conforme os princípios de separação de responsabilidades (Martin, 2020), é neste ambiente isolado e controlado que as requisições da interface são recebidas e processadas. Para sustentar os dados, toda a base é persistida de forma isolada em um banco de dados relacional. Como elucidam Elmasri e Navathe (2011), esse isolamento na infraestrutura de armazenamento protege a integridade e a disponibilidade das informações, consolidando um fluxo perfeito onde o servidor executa consultas estruturadas rápidas e seguras para alimentar as predições matemáticas.

2.4.2 Bancos de dados relacionais

Para viabilizar uma base de conhecimento estruturada, a persistência de sistemas de informação fundamenta-se no modelo de bancos de dados relacionais (Elmasri; Navathe, 2011). Conforme definem Elmasri e Navathe (2011), um banco de dados atua como uma coleção logicamente coerente de dados com significado inerente, projetada e construída para representar fielmente algum aspecto do mundo real. Nos sistemas relacionais, essa organização ocorre por meio da representação dos dados em uma coleção de relações, que se assemelham visualmente a tabelas de valores compostas por linhas e colunas inter-relacionadas (Elmasri; Navathe, 2011).

No ecossistema de bancos relacionais, o PostgreSQL destaca-se como uma das soluções de mercado *open source* mais robustas (Postgres, 2026). De acordo com a documentação oficial do The PostgreSQL Global Development Group (2026), trata-se de um avançado sistema de gerenciamento de banco de dados objeto-relacional de código aberto (*open source*), amplamente reconhecido por suportar grande parte do padrão SQL e oferecer recursos modernos para consultas complexas e integridade transacional. Sistemas com essa capacidade são essenciais em arquiteturas isoladas, pois suportam o crescimento contínuo de registros sem comprometer a performance computacional (Elmasri; Navathe, 2011). Em operações corporativas de software, SGBDs robustos atuam como repositórios definitivos de históricos de projetos. Esse isolamento seguro garante que servidores lógicos (*backends*) executem consultas com alta performance, extraindo massas de dados necessárias para alimentar lógicas de negócios complexas (Elmasri; Navathe, 2011).

2.4.3 Backend e processamento de dados

A infraestrutura lógica preditiva opera na camada de back-end (Elmasri; Navathe, 2011). Nesse cenário arquitetural, o Python destaca-se como tecnologia central para orquestrar as regras de negócio e a inteligência do sistema (Ramalho, 2022). Para garantir que as interfaces de usuário interajam de forma estruturada com os motores analíticos, recomenda-se a utilização de abordagens como a API REST (*Application Programming Interface*) (Freeman, 2017). A proeminência do Python nesse contexto de servidor deve-se à sua forte capacidade de atuar como uma 'linguagem cola', sendo extremamente eficiente para integrar e orquestrar diferentes componentes e algoritmos matemáticos em um único ecossistema (McKinney, 2022). Aliada à sua sintaxe consistente e altamente expressiva (Ramalho, 2022), a linguagem torna-se um ambiente robusto e ideal para receber as requisições HTTP das interfaces web, capturando e validando as estruturas de dados de entrada antes de submetê-las ao processamento analítico.

Em arquiteturas de software modernas, após o recebimento das informações pela API, o *backend* assume papel analítico mais complexo: o processamento da informação e integrações (Martin, 2020). Para preparar os dados para essas integrações, bibliotecas como o Pandas entram em ação como ferramentas de engenharia de dados fundamentais para limpar, manipular e estruturar as variáveis em formatos tabulares e matrizes matemáticas legíveis para a máquina (McKinney, 2022). Uma vez que os dados são estruturados, o Python atua como a ponte final para algoritmos estatísticos e de regressão. Conforme explicam Raschka e Mirjalili (2019), o ecossistema Python permite que modelos de *machine learning* — como os construídos na biblioteca Scikit-Learn — sejam facilmente acoplados e incorporados dentro de aplicações web. Nesse formato arquitetural, o motor analítico recebe os parâmetros de entrada, processa as evidências baseadas em padrões históricos, calcula a inferência estatística e devolve a predição para a API, que empacota e retorna o resultado ao *frontend*, concluindo um fluxo de dados totalmente automatizado (Martin, 2020).

2.4.4 Frontend e tipagem estática

No desenvolvimento da camada de *frontend*, o *framework* Angular ganha destaque por oferecer uma arquitetura escalável baseada em componentes (Freeman, 2017). De acordo com Freeman (2017), o Angular utiliza o padrão de projeto Model-View-Controller (MVC) no lado do cliente para aprimorar os elementos HTML do navegador, permitindo a criação de aplicações

web ricas, modulares e de fácil manutenção. Em plataformas analíticas e gerenciais, essa abordagem é fundamental para organizar a interface em blocos funcionais isolados, para que formulários complexos de entrada e a exibição de resultados operem de maneira altamente reativa (Freeman, 2017). Essa reatividade é impulsionada pelos recursos de *data binding* (associação de dados) nativos do *framework*, que sincronizam instantaneamente os parâmetros inseridos pelo usuário com o modelo de dados da aplicação, garantindo uma navegação em página única (*Single Page Application*) sem exigir recarregamentos a cada requisição processada pelo servidor (Freeman, 2017).

Para fornecer o alicerce lógico de interfaces dinâmicas, o desenvolvimento moderno de aplicações para a camada cliente é orquestrado utilizando o TypeScript (Cherny, 2019). Embora a web funcione tradicionalmente com JavaScript — uma linguagem de tipagem dinâmica e fraca que avalia e converte dados apenas no momento da execução —, essa flexibilidade pode introduzir falhas severas e imprevisíveis em sistemas que lidam com cálculos e regras de negócio rígidas, um risco mitigado pelo TypeScript, que atua como uma linguagem que compila para o próprio JavaScript, mas adiciona o rigor da tipagem estática ao código-fonte (Cherny, 2019). Conforme explica Cherny (2019), a inserção desse sistema de tipos age como uma camada de segurança vital, pois o compilador conta com um analisador estático (*typechecker*) rigoroso para barrar operações inválidas e inconsistências antes mesmo do código ser convertido e executado no navegador. No desenvolvimento de softwares confiáveis, essa tipagem garante que as estruturas de dados e as variáveis respeitem contratos estruturais exatos (Cherny, 2019). Consequentemente, essa restrição estrutural blinda o fluxo operacional e assegura que o sistema processe apenas informações em seu formato perfeitamente tipado e validado, transformando em erros de compilação a grande maioria das falhas que ocorreriam no tempo de execução (Cherny, 2019).

2.5 TRABALHOS CORRELATOS

A estimativa de esforço no desenvolvimento de software é um desafio crítico para o planejamento e a alocação de recursos. Para superar as limitações das estimativas baseadas apenas no julgamento de especialistas, Corrêa (2020) propôs uma metodologia focada no Aprendizado de Máquina Automatizado (AutoML). O objetivo principal da pesquisa foi reduzir a necessidade de parametrização manual dos algoritmos, facilitando o uso de inteligência artificial por engenheiros e profissionais sem expertise profunda em ciência de dados. Em seu estudo, Corrêa (2020) avaliou duas ferramentas de AutoML (*Auto-Sklearn* e TPOT) aplicando-

as em dez *datasets* distintos, analisando a eficácia preditiva através das métricas MAE, MdMRE, MMRE e PRED(25). Os resultados demonstraram que a metodologia automatizada consegue se autoajustar e generalizar estimativas consistentes para múltiplos conjuntos de dados de forma promissora, mantendo as margens de erro relativo e as taxas de predição aceitáveis dentro dos limites satisfatórios recomendados na área.

Avançando na investigação empírica sobre a eficiência dos algoritmos, Caldeira (2025) conduziu uma avaliação comparativa de modelos supervisionados, testando a Regressão Linear, SVR, Árvore de Decisão, Random Forest e Gradient Boosting em seis *datasets* públicos. A partir do cálculo de métricas de precisão como RMSE e R^2, os resultados indicaram que os algoritmos baseados em árvores (Random Forest e Gradient Boosting) demonstraram uma estabilidade preditiva significativamente maior em bases heterogêneas e com alta dispersão de esforço. O autor constatou empiricamente que a Regressão Linear foi competitiva apenas em bases que se aproximam do padrão linear, falhando e obtendo um coeficiente de determinação (R^2) severamente negativo no dataset NASA93 devido à assimetria e à presença de poucos casos de alto esforço (valores extremos). Por fim, o estudo também observou um comportamento instável no algoritmo SVR, concluindo que modelos flexíveis e baseados em conjuntos (*ensembles*) oferecem maior robustez em relação aos baselines lineares simples para lidar com a complexidade intrínseca dos dados de projetos reais.

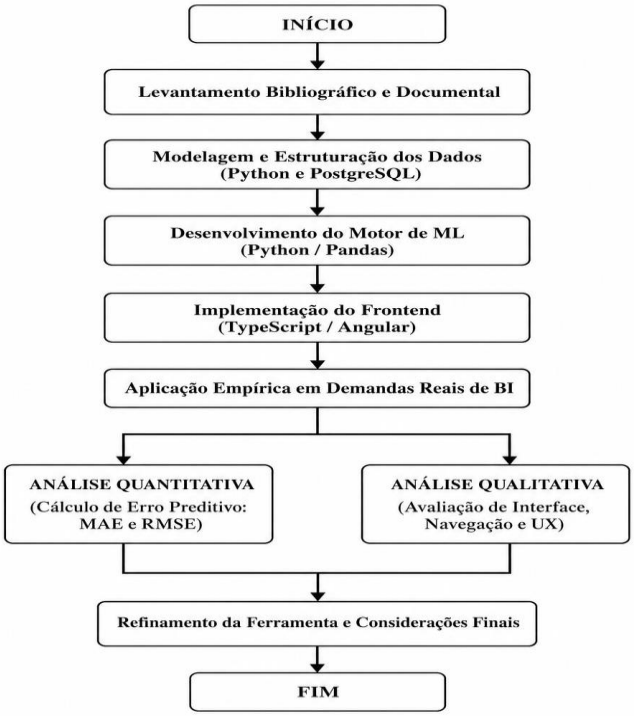
3. METODOLOGIA DA PESQUISA

O presente Trabalho de Conclusão de Curso caracterizar-se-á como uma pesquisa aplicada e descritiva, pois seu objetivo será desenvolver uma ferramenta prática e funcional em formato de sistema web que permita aos usuários, como pequenas empresas, estimar horas e custos de demandas de b*usiness intelligence*, como o desenvolvimento de *dashboards* com *Power BI* e *reports* com python, e análise de dados com base em dados históricos e inteligência artificial.

O trabalho buscará responder aos seguintes questionamentos: De que maneira a substituição do palpite e da intuição humana por modelos preditivos de machine learning impacta a precisão dos orçamentos de software? Como visualizar de forma clara e acessível as comparações entre o esforço real e o estimado? Quanto aos procedimentos utilizados, recorrer-se-á ao levantamento bibliográfico e documental, com foco em fontes reais sobre engenharia de software, métodos de estimativa, inteligência artificial e tecnologias de desenvolvimento de software, além de um teste prático de usabilidade com usuários reais para validação da ferramenta desenvolvida.

A pesquisa será operacionalizada da seguinte maneira: Inicialmente, será realizada a pesquisa bibliográfica para embasar teoricamente os temas envolvidos no desenvolvimento do sistema, com destaque para a linguagem Python e a biblioteca Pandas para os algoritmos de regressão, o banco de dados PostgreSQL e o *framework* Angular. Em seguida, a ferramenta será desenvolvida utilizando essas tecnologias, permitindo ao usuário inserir informações como escopo da demanda, horas trabalhadas no passado e variáveis técnicas específicas da área de dados (como complexidade de integrações e nível de consultas SQL). A partir dessas entradas, o sistema preditivo será projetado para retornar a estimativa quantitativa de horas e custos da nova demanda, apresentando os valores comparativos que facilitarão o acompanhamento gerencial do projeto.

Figura 1 – Fluxograma do processo do desenvolvimento do sistema



Fonte: Elaborado pelo autor (2026).

Para a operacionalização do teste prático mencionado, a ferramenta será aplicada de forma empírica no próprio ambiente de trabalho do pesquisador. Essa etapa consistirá na utilização rotineira do software para auxiliar a orçar demandas reais de b*usiness intelligence*, a fim de avaliar a clareza da interface, a eficiência da navegação e a precisão dos resultados preditivos gerados pelo algoritmo. A partir dessa experiência de uso direto, os dados e as percepções obtidas serão analisados de duas formas: quantitativa, comparando os resultados matemáticos de erro preditivo (como MAE e RMSE), e qualitativa, com foco no refinamento da ferramenta e na melhoria contínua da experiência do usuário.

4. RESULTADOS DO TC (RECOMENDA-SE USAR O TÍTULO DO TC AQUI)

5. CONCLUSÃO

REFERÊNCIAS

BLOCH, Michael; BLUMBERG, Sven; LAARTZ, Jürgen. **Delivering large-scale IT projects on time, on budget, and on value**. McKinsey on Business Technology, [s. l.], out. 2012.

BROOKS JR., Frederick P. **The mythical man-month:** essays on software engineering**.** Boston: Addison-Wesley, 1995.

CALDEIRA, Victor Menezes. **Avaliação da Eficiência de Algoritmos de Machine Learning para Estimativa de Esforço de Software em Datasets Públicos**. 2025. 16 f. Trabalho de Conclusão de Curso (Bacharelado em Engenharia de Software) - Pontifícia Universidade Católica de Minas Gerais (PUC Minas), Belo Horizonte, 2025.

CHERNY, Boris. **Programming TypeScript:**making your JavaScript applications scale**.** Sebastopol: O'Reilly Media, 2019.

CORRÊA, Weldson Amaral. **Aplicando Aprendizado de Máquina para Estimativa de Esforço no Desenvolvimento de Software**. 2020. 66 f. Dissertação (Programa de Pós-Graduação em Ciência da Computação/CCET) - Universidade Federal do Maranhão, São Luís, 2020.

ELMASRI, Ramez; NAVATHE, Shamkant B. **Sistemas de banco de dados.** São Paulo: Pearson Addison Wesley, 2011.

FREEMAN, Adam. **Pro Angular.** New York: Apress, 2017.

HASTIE, Trevor; JAMES, Gareth M.; TIBSHIRANI, Robert; WITTEN, Daniela. **An introduction to statistical learning:** with applications in R. New York: Springer, 2013.

MARTIN, Robert Cecil. **Arquitetura limpa:** o guia do artesão para estrutura e design de software. Rio de Janeiro: Alta Books, 2020.

MCCONNELL, Steve. **Software estimation:** demystifying the black art. Redmond: Microsoft Press, 2006.

MCKINNEY, Wes. **Python for data analysis**. Sebastopol: O'Reilly Media, 2022.

POSTGRES. **PostgreSQL 18 Documentation**. [S. l.], 2026. Disponível em: https://www.postgresql.org/files/documentation/pdf/18/postgresql-18-A4.pdf. Acesso em: 9 jun. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. **Engenharia de software:** uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

RAMALHO, Luciano. **Fluent Python:** clear, concise, and effective programming**.** Sebastopol: O'Reilly Media, 2022.

RASCHKA, Sebastian; MIRJALILI, Vahid. **Python machine learning:** machine learning and deep learning with Python, scikit-learn, and TensorFlow 2. Birmingham: Packt Publishing, 2019.

RUSSELL, Stuart; NORVIG, Peter. J. **Artificial intelligence:** a modern approach. Hoboken: Pearson, 2021.

THE STANDISH GROUP. **The Standish Group Report**: CHAOS. [s. l.]: The Standish Group, 1995.

ANEXOS